

PHASE 1: REGISTRATION (Done once at the bank)

Step R1: User Provides Identity to Bank Officer

Input:

- Sender = A
- NID = 19XX-XXXX-XXXX-XXXX
- Activation Code (from bank officer) = ACT-9981
- MAC Address = AA:BB:CC:DD:EE:FF
- Biometric = Fingerprint (BP)
- Password (K2) = user-chosen, only known by user

Step R2: Fingerprint Converted to Secure Hash

Operation:

- BP_hash = SHA-256(Fingerprint)
- = 9f86d081884c7d659a2feaa0c55ad015a3bf4f1

Explanation: Raw biometric data is never stored. A one-way hash is taken so the original fingerprint cannot be exposed or reused. This also fixes the non-revocability problem of biometric data.

Step R3: Generate K1 — HMAC Key (Paper's Corrected Version)

Operation:

- $K1 = \text{HMAC-SHA256}(\text{ACT-9981} \parallel \text{NID}, \text{MAC_Address} \parallel \text{BP_hash})$
- = $\text{HMAC-SHA256}(\text{'ACT-998119XX-XXXX-XXXX-XXXX'}, \text{'AA:BB:CC:DD:EE:FF9f86d081...'})$
- = K1_c7f3a9b82d14e056...
- → K1 is stored securely at the bank server (linked to user account)

Step R4: Stretch Password K2 Using PBKDF2

Operation:

- K2_stretched = PBKDF2(K2_password, NID_as_salt, iterations=100000)
- = PBKDF2('userpass', '19XX-XXXX', 100000)
- = 5e884898da28047151d0e56f8dc62927... (256-bit)
- → K2_stretched stored at server. Raw password is NEVER stored.

Explanation: PBKDF2 with 100,000 iterations makes brute-force attacks computationally infeasible.

Step R5: Initialize Last_T in Database

Operation:

After registration is complete: Last_T = 0 (initial value)

→ Store in the database: User Account = A

→ Last_T = 0

→ This means for the user's first transaction, any valid timestamp T will pass Check 2 because any real $T > 0$

Explanation:

At the time of registration, Last_T is set to 0 in the database for the user. This acts as a placeholder, indicating that the user has not performed any transaction yet. In the first transaction, $T > 0$ will always be true, so it will pass. After that, for every successful transaction, this value will be updated according to Step 13.

PHASE 2: TRANSACTION — SENDER SIDE (User's Phone)

Step 1: User Starts a Transaction

Input:

- Sender = A
- Receiver = B
- Amount = 1000
- Password = userpass (entered by user)
- Biometric = Fingerprint (scanned at time of transaction)

Explanation: User logs in with username + password + fingerprint. All three must match — this is multi-factor authentication. No single factor alone is enough.

Step 2: Fingerprint Hashed Again (Same as Registration)

Operation:

- BP_hash = SHA-256(Fingerprint)
- = 9f86d081884c7d659a2feaa0c55ad015a3bf4f1

Explanation: Biometric is re-hashed fresh at every transaction. The hash is used in key derivation — raw fingerprint is never transmitted or stored.

Step 3: Construct AES Encryption Key

Operation:

- T (Timestamp) = 1715600000 (current Unix time)
- AES_Key = HMAC-SHA256(K2_stretched , (BP_hash | T))

- = HMAC-SHA256('5e884898...', '9f86d081...' + '1715600000')
- = 2c26b46b68ffc68ff99b453c1d304134... (256-bit AES key)

Explanation: Timestamp T is embedded so the key changes every transaction.

Step 4: Create Transaction Message M

Operation:

- $M = \{ \text{Receiver: B, Amount: 1000, Timestamp: 1715600000} \}$

Explanation: Message M contains receiver, amount, and timestamp. The timestamp ensures the message is time-bound — old messages cannot be reused (replay prevention).

Step 5: Compute $F1 = \text{HMAC}(K1, M)$ — Integrity Hash

Operation:

- $F1 = \text{HMAC-SHA256}(K1, M)$
- = HMAC-SHA256('K1_c7f3a9b82d14e056...', '{Receiver:B, Amount:1000, Time:1715600000}')
- = F1_98f13708210194c475687be6106a3b84

Explanation: F1 is the integrity seal of the message. Any change to M — even one character — will produce a completely different F1. K1 also binds the message to this specific device since MAC address was used to build K1.

Step 6: Combine M and F1

Operation:

- Payload = M | F1
- = '{Receiver:B, Amount:1000, Time:1715600000}' + '|' + 'F1_98f13708210194c475687be6106a3b84'

Explanation: Message and its integrity code are joined before encryption.

Step 7: Encrypt Payload Using AES-256

Operation:

- IV (Initialization Vector) = a1b2c3d4e5f6a7b8 (random, new each time)
- Encrypted_Data = AES-256-CBC(AES_Key , IV , Payload)
- = 6f1ed002ab5595859014ebf0951522d9... (ciphertext)

Explanation: AES-256 encryption locks the combined payload. A fresh random IV is generated every transaction so the same message never produces the same ciphertext. This prevents pattern analysis by attackers.

Step 8: Send Packet to Bank Server

Transmitted Packet:

- IV = a1b2c3d4e5f6a7b8
- Encrypted_Data = 6f1ed002ab5595859014ebf0951522d9...
- Timestamp (T) = 1715600000
- → Password, fingerprint, K1, K2 are NEVER transmitted.

Explanation: Only the encrypted ciphertext, IV, and timestamp are sent. An attacker intercepting this channel sees only random-looking bytes. All secret keys remain on the device and server.

PHASE 3: TRANSACTION — SERVER SIDE (Bank Server)

Step 1: Timestamp Validation — Replay Attack Prevention

Operation:

- Received T = 1715600000
- Server current time = 1715600120
- Last_T from Database = 1715599500 (previous successful transaction)
- Check 1: $|Current_Time - T| \leq 180$ seconds?
- $|1715600120 - 1715600000| = 120$ seconds → PASS
- Check 2: T > Last_T stored in Database?
- $1715600000 > 1715599500$ → PASS
- Both checks passed → Proceed to Step 2
- If Check 1 fails → Request too old → REJECT
- If Check 2 fails → Replay Attack detected → REJECT

Explanation: Two-layer replay protection. Check 1 ensures the request is recent within a 3-minute window. Check 2 ensures this exact timestamp was never used before by comparing with the last stored T in the database. Even if an attacker captures and resends the packet within 5 minutes, Check 2 will catch it because the same T cannot be used twice.

Step 2: Reconstruct AES Key at Server

Operation:

- Server has stored: K2_stretched, BP_hash, K1 (from registration)
- AES_Key = HMAC-SHA256(K2_stretched , (BP_hash | T))
- = HMAC-SHA256('5e884898...' , '9f86d081...' + '1715600000')
- = 2c26b46b68ffc68ff99b453c1d304134...

Explanation: Server independently recreates the same AES key using the same formula. This is only possible if the correct password, biometric, and timestamp were used — confirming user identity without transmitting any secret.

Step 3: Decrypt the Ciphertext

Operation:

- Payload = AES-256-Decrypt(AES_Key , IV , Encrypted_Data)
- = '{Receiver:B, Amount:1000, Time:1715600000} | F1_98f13708...'
- Extract: M = {Receiver:B, Amount:1000, Time:1715600000}
- Extract: F1 = F1_98f13708210194c475687be6106a3b84

Explanation: Server decrypts the packet to recover M and F1. If decryption fails or produces garbage, the packet was tampered — transaction aborted.

Step 4: Verify Integrity — Recompute F2 and Compare

Operation:

- F2 = HMAC-SHA256(K1_stored , M)
- = HMAC-SHA256('K1_c7f3a9b82d14e056...' , '{Receiver:B, Amount:1000, Time:1715600000}')
- = F1_98f13708210194c475687be6106a3b84
- Check: F1 == F2?
- F1_98f13708... == F1_98f13708... → MATCH — Message Accepted
- If F1 ≠ F2 → Message was tampered → REJECT

Explanation: Server recomputes the HMAC using K1 stored at registration. If F1 matches F2, it confirms: (1) message was not modified, (2) it came from the correct registered device, since K1 contains the MAC address.

Step 5: Process Transaction

Operation:

- Receiver B found in database? → YES
- Sender A balance >= Amount 1000? → YES
- → Deduct 1000 from Sender A
- → Add 1000 to Receiver B
- → Update Database: Last_T = 1715600000 (store this transaction's T)
- → In next transaction if T is not bigger then Last_T then REJECT
- → Send success notification to both A and B
- If balance insufficient → Reject, notify sender only.

Explanation: After successful transfer, Last_T is updated in the database to the current transaction's T. This means the next transaction must carry a strictly larger T value. This is the actual cryptographic purpose of the paper's statement "timestamp is updated each time a transaction is successful." Combined with Step 9's double check, replay attacks are completely eliminated.

FINAL RESULT

TRANSACTION SUCCESSFULLY VERIFIED, SECURE, AND COMPLETED.

The system ensures Confidentiality (AES-256), Integrity (HMAC $F1=F2$), Authentication (K1 device binding), and Replay Prevention (Timestamp window).